

C++&Qt Day01 hw2 보고서

작성자: Doyeon Park

작성일: 2026.09.03

목차

1. 코드 구성
2. class 및 구조체 설계
3. 주요 함수 설계
4. 코드 전체 동작 과정
5. 실행 결과

1. 코드 구성

기본적인 파일 구조는 다음과 같다

```
hw2/  
hw2/main.cpp  
hw2/hw2.hpp  
hw2/hw2.cpp
```

각 파일의 역할은 다음과 같다.

main.cpp	→	객체 생성 및 전체 프로그램 실행 순서 담당
hw2.hpp	→	Point 구조체와 PointDistance 클래스 선언
hw2.cpp	→	PointDistance 클래스의 멤버함수 구현

2. class 및 구조체 설계

2-1 Point 구조체

```
struct Point  
{  
    // x  
    int x;  
    // y  
    int y;  
};
```

하나의 점에는 x, y 두 좌표가 필요하다. 이 두 값을 하나의 자료형으로 묶기 위해 struct Point 를 사용하였다. 이후 Point* points 를 이용해서 여러 점을 배열로 관리하였다.

2-2 PointDistance class

멤버변수를 정리하면 다음과 같다.

변수	→	역할
count	→	생성할 점의 개수
minCoord	→	좌표 최솟값
maxCoord	→	좌표 최댓값
Point* points	→	동적으로 생성한 점 배열
minDistance	→	최소거리
maxDistance	→	최대거리
minIndex1, minIndex2	→	최소 거리인 두 점의 인덱스
maxIndex1, maxIndex2	→	최대 거리인 두 점의 인덱스

* 과제 1에서는 int* arr; 였지만 이번 과제에선 정수가 아니라 Point 객체를 여러 개 저장하기 때문에 Point*를 사용하였다.

3. 주요 함수 설명

3-1 생성자/소멸자

생성자에서 멤버 변수를 초기화 하고 소멸자에선 delete[] points; 로 new[]로 생성한 배열을 해제하였다.

```
PointDistance::PointDistance()
{
    count = 0;
    minCoord = 0;
    maxCoord = 0;
    points = nullptr;
    minDistance = 0;
    maxDistance = 0;
    minIndex1 = 0;
    minIndex2 = 0;
    maxIndex1 = 0;
    maxIndex2 = 0;
}

PointDistance::~PointDistance()
{
    delete[] points;
}
```

3-2 input()

사용자에게 점 개수, 최소 좌표, 최대 좌표를 입력받는 함수이다.
입력한 count 만큼 Point 배열을 동적 생성한다.

```

void PointDistance::input()
{
    cout << "Please define the number of points: ";
    cin >> count;

    cout << "Please define minimum of coor. value: ";
    cin >> minCoord;
    cout << "Please define maximum of coor. value: ";
    cin >> maxCoord;

    points = new Point[count];
}

```

3-3 generatePoints()

이 함수는 사용자가 랜덤 범위를 input()에서 작성하면 그 값을 받아 실제로 사용자가 정의한 그 사이 값 중 랜덤한 값을 만들어내는 함수이다. C++ rand()함수를 이용하였으며 코드는 다음과 같다.

```

void PointDistance::generatePoints()
{
    for (int i = 0; i < count; i++)
    {
        points[i].x = rand() % (maxCoord - minCoord + 1) + minCoord;
        points[i].y = rand() % (maxCoord - minCoord + 1) + minCoord;
    }
}

```

예를 들어 minCoord = 10, maxCoord = 20 이면 rand() % 11 + 10 으로 동작하여 범위가 10~20 으로 생성되는 방식이다.

3-4 printPoints()

for (int i = 0; i < count; i++) 을 이용하여 생성된 모든 점을 출력하는 함수이다.

points[i].x 와 points[i].y 가 각각 i 번째 Point 의 x/y 좌표이다.

3-5 calculate()

여기서는 이중 반복문을 사용한다. 먼저 코드를 살펴보면 다음과 같다.

```
void PointDistance::calculate()
{
    for (int i = 0; i < count; i++) {
        for (int j = i + 1; j < count; j++) {
            double distance = sqrt(
                (points[i].x - points[j].x) * (points[i].x - points[j].x)
                +
                (points[i].y - points[j].y) * (points[i].y - points[j].y)
            );
            if (i == 0 && j == 1) {
                minDistance = distance;
                maxDistance = distance;

                minIndex1 = i;
                minIndex2 = j;

                maxIndex1 = i;
                maxIndex2 = j;
            }
            else
            {
                if (distance < minDistance) {
                    minDistance = distance;
                    minIndex1 = i;
                    minIndex2 = j;
                }

                if (distance > maxDistance) {
                    maxDistance = distance;
                    maxIndex1 = i;
                    maxIndex2 = j;
                }
            }
        }
    }
}
```

예를 들어 점이 4 개인 경우 0-1, 0-2, 0-3, 1-2,

1-3, 2-3 만 계산한다.

또한 두 점 사이 유클리드 거리공식을 C++코드로 구현하였다.

처음에는 실제 거리의 최솟값과 최댓값을 모르므로 첫번째로 계산한 거리를 우선 min, maxDistance 에 저장한다. 이후 새로운 거리를 계산하면 그때 최소 최대를 기존 min, maxDistance 와 비교하여 알맞는 값으로 교체한다. 또한 거리만 저장하는것이 아닌 minIndex1 = i; 와 minIndex2 = j;를 이용하여 어떤 두 점이었는지도 같이 저장한다.

3-6 printResult()

이 함수는 결과를 출력하는 함수이다.

출력하는 내용은 다음과 같다.

최소 거리 출력

해당하는 두 점의 번호와 좌표출력

최대 거리 출력

해당하는 두 점의 번호와 좌표 출력

여기서 `minIndex1 + 1` 을 쓰는 이유는 배열 index 는 0, 1, 2 ... 이지만,
사용자가 인식하는 점 번호는 1, 2, 3... 이기 때문이다.

4. 코드 전체 동작 과정

코드의 흐름은 다음과 같다.

프로그램 시작 → PointDistance 객체 생성 → 점 개수 / 좌표 범위 입력 → Point 배열 동적 할당
→ 랜덤 좌표 생성 → 생성된 점 출력 → 모든 두 점 사이 거리 계산 → 최소/최대 거리 탐색 → 해당 두 점
및 거리 출력 → 소멸자 실행 → 동적 메모리 해제 → 프로그램 종료

5. 실행 결과

```
• doyeon@doyeon-park:~/VScode/C++&Qt/Day01/hw2$ g++ main.cpp hw2.cpp -o hw2
• doyeon@doyeon-park:~/VScode/C++&Qt/Day01/hw2$ ./hw2
Please define the number of points: 10
Please define minimum of coord. value: 0
Please define maximum of coord. value: 20
Point 1 : (11, 12)
Point 2 : (13, 17)
Point 3 : (10, 16)
Point 4 : (3, 14)
Point 5 : (6, 3)
Point 6 : (18, 4)
Point 7 : (3, 4)
Point 8 : (20, 14)
Point 9 : (17, 10)
Point 10 : (5, 10)
-----result-----
Minimum distance : 3.16228
Point 2: (13, 17)
Point 3: (10, 16)
Maximum distance : 19.7231
Point 7: (3, 4)
Point 8: (20, 14)
○ doyeon@doyeon-park:~/VScode/C++&Qt/Day01/hw2$
```